

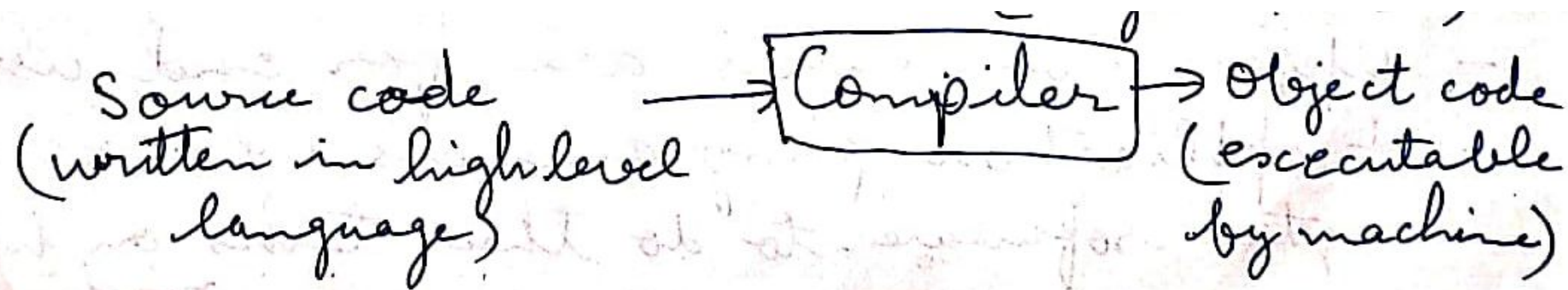
Foundations of System Programming

By Dr. Satinder Kaur

Introduction

In the early days of computers, people communicated with them by *on* and *off* switches denoting primitive instructions. Soon people wanted to give more complex instructions. For example, they wanted to be able to say $X = 30 * Y$; given that $Y = 10$, what is X ? Present day computers cannot understand such language without the aid of systems programs. Systems programs (e.g., compilers, loaders, macro processors, operating systems) were developed to make computers better adapted to the needs of their users. Further, people wanted more assistance in the mechanics of preparing their programs.

Compilers are systems programs that accept people-like languages and translate them into machine language. Loaders are systems programs that prepare machine language programs for execution. Macro processors allow programmers to use abbreviations. Operating systems and file systems allow flexible storing and retrieval of information (Fig. 1.1).



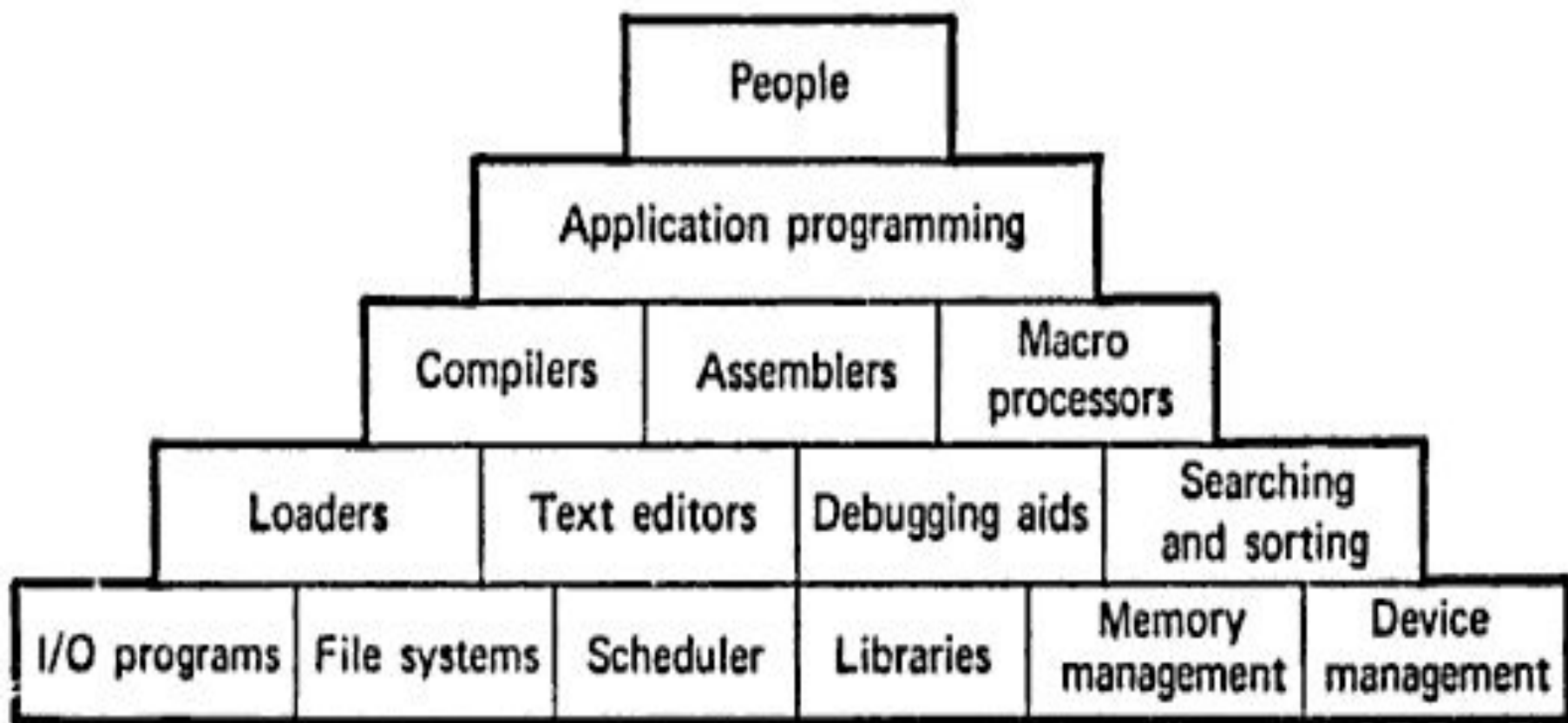


FIGURE 1.1 Foundations of systems programming

Assemblers

Programmers found it difficult to write or read programs in machine language. In their quest for a more convenient language they began to use a *mnemonic* (symbol) for each machine instruction, which they would subsequently translate into machine language. Such a mnemonic machine language is now called an *assembly language*. Programs known as *assemblers* were written to automate the translation of assembly language into machine language. The input to an assembler program is called the *source program*; the output is a machine language translation (*object program*).

Loader

A *loader* is a program that places programs into memory and prepares them for execution. In a simple loading scheme, the assembler outputs the machine language translation of a program on a secondary storage device and a loader is placed in core. The loader places into memory the machine language version of the user's program and transfers control to it. Since the loader program is much smaller than the assembler, this makes more core available to the user's program.

Macroprocessor

operating systems provide a macro processing facility, which permits the programmer to define an abbreviation for a part of his program and to use the abbreviation in his program. The macro processor treats the identical parts of the program defined by the abbreviation as a *macro definition* and saves the definition. The macro processor substitutes the definition for all occurrences of the abbreviation (macro call) in the program.

Operating Systems

An operating system is concerned with the allocation of resources and services, such as memory, processors, devices, and information. The operating system correspondingly includes programs to manage these resources, such as a *traffic controller*, a *scheduler*, *memory management module*, *I/O programs*, and a *file system*.

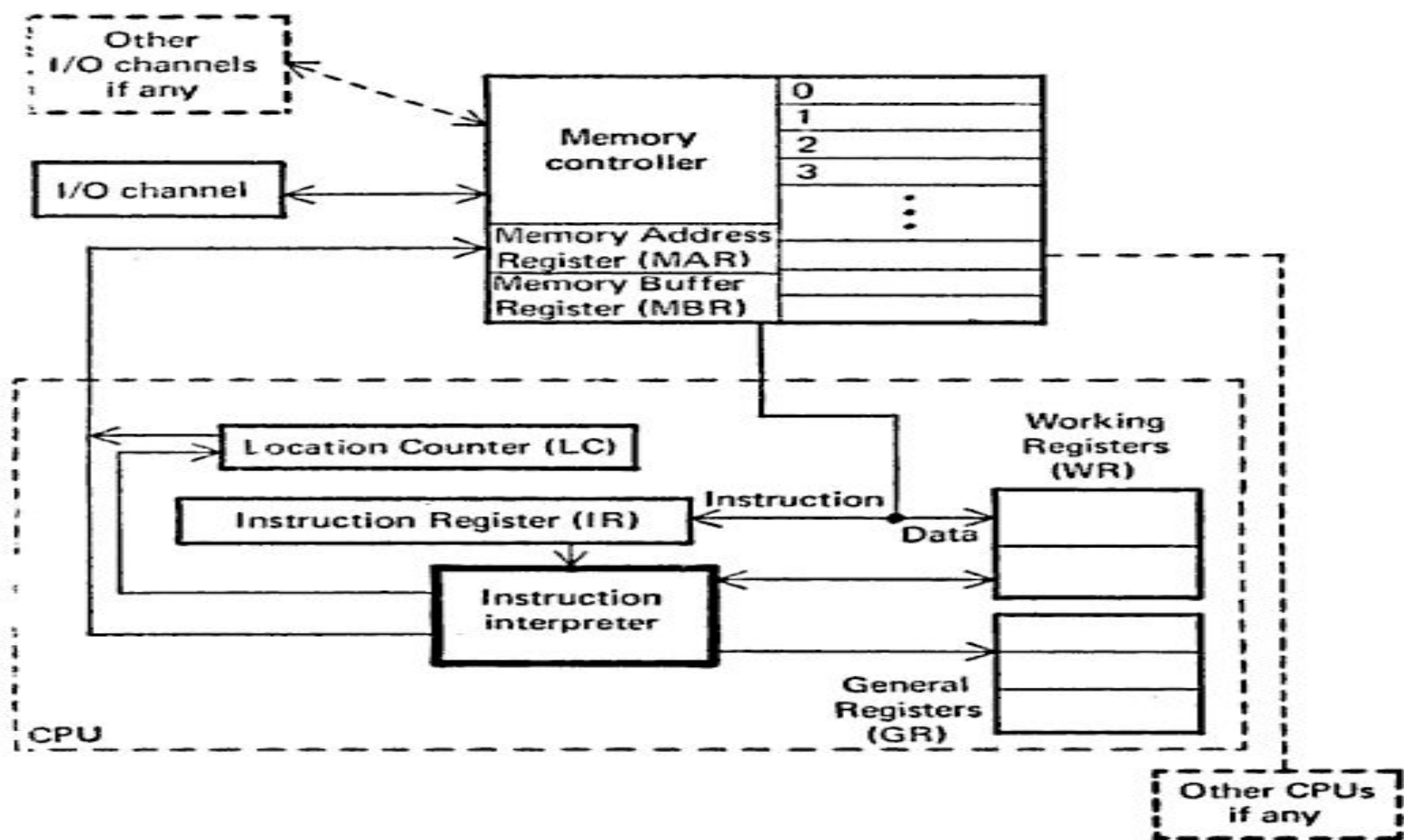
General Machine Structure

Almost all conventional modern computers are based upon the “stored program computer” concept, generally credited to the mathematician John von Neumann (1903-1957). Figure 2.1 illustrates the structure of the CPU for a typical von Neumann machine, such as the IBM System/360.

The CPU consists of an instruction interpreter, a location counter, an instruction register and various working registers and general registers. The *instruction interpreter* is a group of electrical circuits (*hardware*), that performs the intent of instructions fetched from memory. The *Location Counter* (LC), also called

Program Counter (PC) or *Instruction Counter (IC)*, is a hardware memory device which denotes the location of the current instruction being executed. A copy of the current instruction is stored in the *Instruction Register (IR)*. The *working registers* are memory devices that serve as “scratch pads” for the instruction interpreter, while the *general registers* are used by the programmer as storage locations and for special functions.

The primary interface between the memory and the CPU is via the memory address register and the memory buffer register. The *Memory Address Register (MAR)* contains the address of the memory location that is to be read from or stored into. The *Memory Buffer Register (MBR)* contains a copy of the designated memory location specified by the MAR after a “read,” or the new contents of the memory location prior to a “write.” The *memory controller* is hardware



For example, the instruction

ADD 2,176

would cause the data stored in memory location 176 to be added to the current contents of general register 2. The resulting sum would be left as the new contents of register 2. The micro-flowchart in Figure 2.2 illustrates the sequence of hardware operations performed within the instruction interpreter to execute an instruction.

Although the specific details vary from computer to computer, this example of machine structure is representative of all conventional machines.

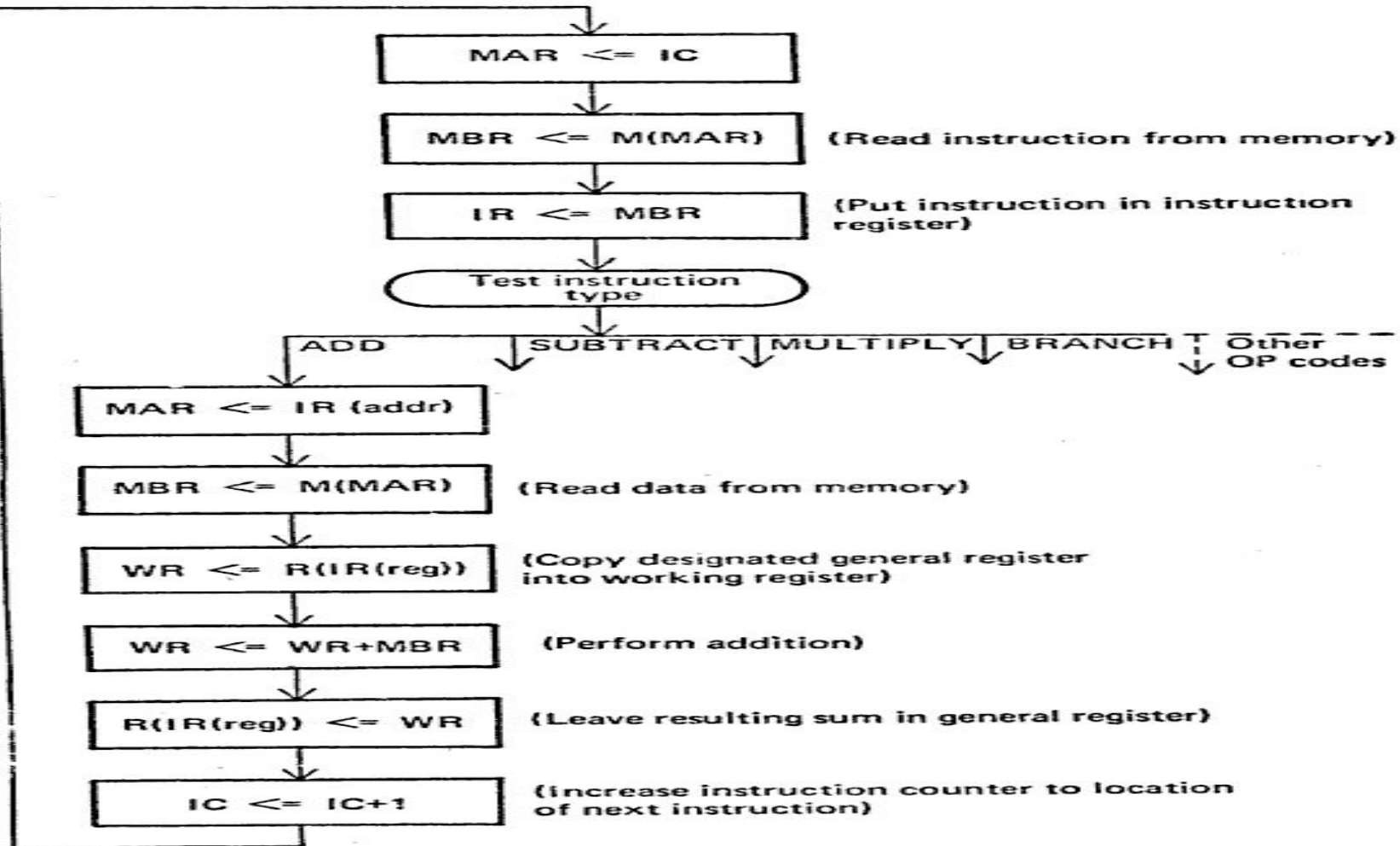


FIGURE 2.2 Example micro flowchart for ADD instruction

Questions for machine architecture

1. MEMORY

What is the memory's basic unit, size, and addressing scheme?

2. REGISTERS

How many registers are there? What are their size, function, and interrelationship?

3. DATA

What types of data can be handled by the computer? Can it handle characters, numbers, logical data? How is this data stored?

4. INSTRUCTIONS

What are the classes of instructions on the machine? Are there arithmetic instructions, logical instructions, symbol-manipulation instructions? What are their formats? How are they stored in memory?

5. SPECIAL FEATURES

What is the interrupt structure of the machine? What sort of protection mechanism is available to the user?

IBM 360/370

1. MEMORY

The basic unit of memory in the 360 is a byte – eight bits of information. That is, each addressable position in memory can contain eight bits of information. There are facilities to operate on contiguous bytes in basic units. The basic units are as follows:

<i>Unit of memory</i>	<i>Bytes</i>	<i>Length in bits</i>
Byte	1	8
Halfword	2	16
Word	4	32
Doubleword	8	64

A unit of memory consisting of four bits is sometimes referred to as a *nibble*.

The size of the 360 memory is up to 2^{24} bytes (about sixteen million).

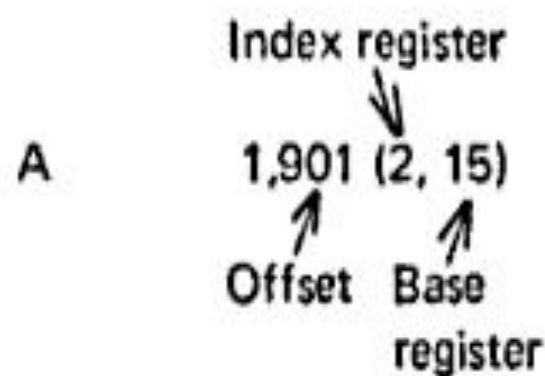
The addressing on the 360 memory may consist of three components. Specifically, the value of an address equals the value of an offset, plus the contents of a base register, plus the contents of an index register. We will give examples of this addressing later.

In general, operations on units of memory are specified by the low-order byte address. For example, when addressing a word (four bytes) the address of the word is that of the low order byte.

2. REGISTERS

The 360 has 16 general-purpose registers consisting of 32 bits each. In addition there are 4 floating-point registers consisting of 64 bits each. It has a 64-bit Program Status Word (PSW) that contains the value of the location counter, protection information, and interrupt status.

The general-purpose registers may be used for various arithmetic and logical operations and as base registers. When the programmer uses them in arithmetic or logical operations, he thinks of these registers as scratch pads to which numbers are added, subtracted, compared, and so forth. When used as base registers, they aid in the formation of the address. Take for example the instruction



It is interpreted as an add instruction. A number is to be added to the contents of register 1.

The location of the number is 901 (*offset*) plus the contents of register 2 (*index*) plus the contents of register 15 (*base*). That is, if those three numbers were added together, the result would be the address of the memory location whose contents we wish to add to the contents of register 1.

One may ask why such complexity in the formation of addressing is necessary. The motivation is twofold. First, a base register aids in the process of relocation of a program. As we will see, an entire program may be moved from one series of locations to another by changing the contents of a base register.² A major motivation for employing base registers, however, is to promote efficient addressing of core. For example, in order to address all possible core locations (16 million) in the 360 without the use of a base register, we would need 24 bits for every address. By way of illustration, if the preceding add instruction were formed in core as depicted in the following diagram, we would need a total of 40 bits to store it: 8 bits for the op code, 4 bits to specify one of 16 possible registers to which the number is added, an additional 4 bits to specify one of 16 possible index registers, and lastly, 24 bits to specify the address of the number we wish to add

Op code	Arg reg	Index reg	Address
---------	---------	-----------	---------

8 bits

4 bits

4 bits

24 bits

Total: 40

If we use a base register, we can store the instruction in the following format. We could specify any one of 16 possible registers as the base register, using 4 bits, and employ an additional 12 for an offset. The total number of bits for an add instruction would be 32, a savings of 8 bits per address reference.

Op code	Arg reg	Index reg	Base	Offset
---------	---------	-----------	------	--------

8 bits

4 bits

4 bits

4 bits

12 bits

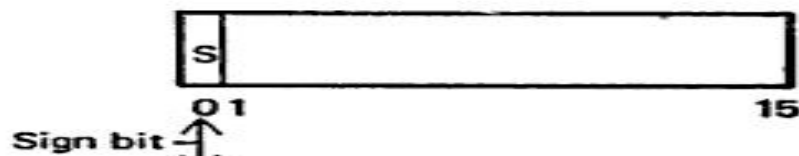
Total: 32

The disadvantages of this shorter form are the overhead associated with the formation of the address during execution and the fact that the offset, which is 12 bits long, can only specify a number from 0 to 4,095. Thus, it may be difficult to "reach" the data. That is, without using an index register and without changing the contents of the base register, the core location we wish to address cannot be any further than 4,095 locations away from the core location to which the base register is pointing.

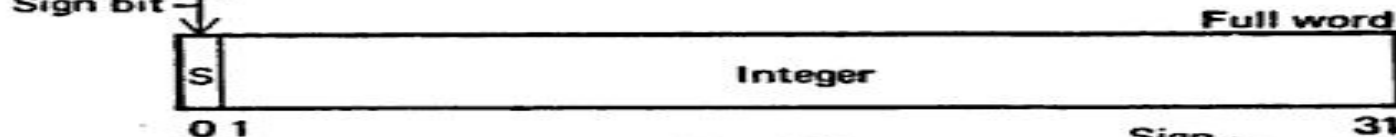
3. DATA

The 360 may store several different types of data as is depicted in Figure 2.3. That is, groups of bits stored in memory are interpreted by a 360 processor in several ways. If a 360 interprets the contents of two bytes as an integer (Fig. 2.3a), it interprets the first bit as a sign and the remaining 15 as a binary number (e.g., 0000 0010 0001 1101 is interpreted as the binary number equivalent to the decimal number +541).³ If a 360 interprets the contents of two bytes as a packed decimal (Fig. 2.3c), it would interpret the first byte as two BCD coded digits, the first four bits of the second byte as a BCD digit, and the last four as a sign (e.g., $\underbrace{0000}_0 \underbrace{0010}_2 \underbrace{0001}_1 \underbrace{1101}_{\text{Sign}}$ is interpreted as the decimal number -021).

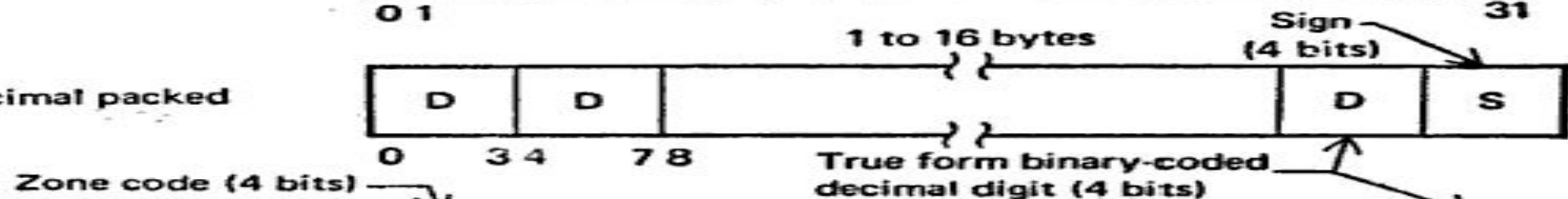
a) Short form fixed point



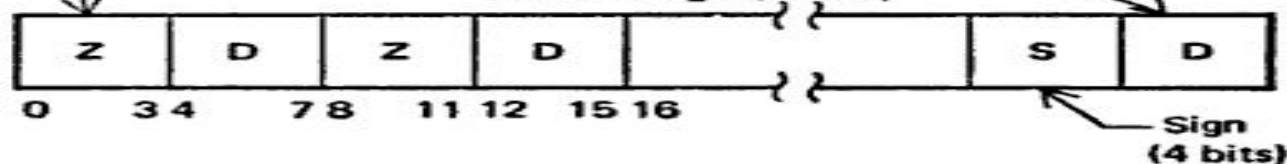
b) Long form fixed point



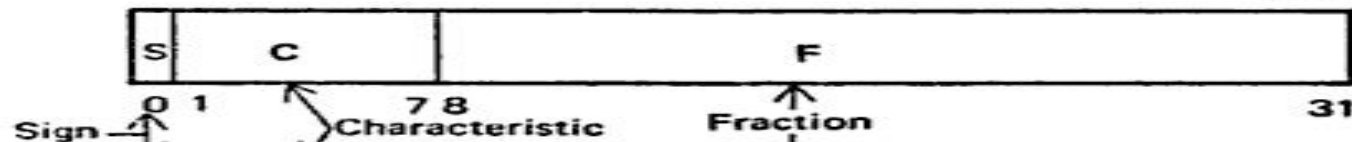
c) Decimal packed



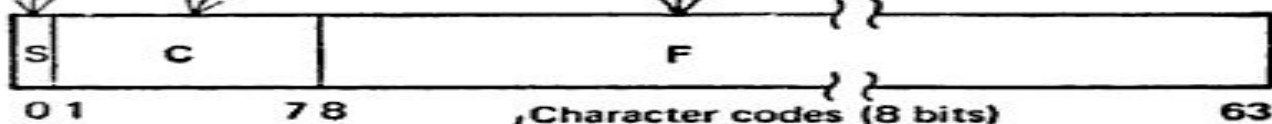
d) Unpacked



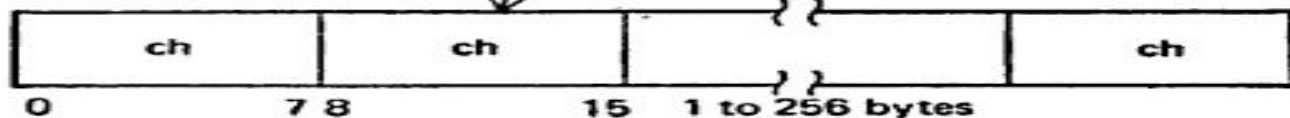
e) Short form floating point



f) Long form floating point



g) Logical (characters)



4. INSTRUCTIONS

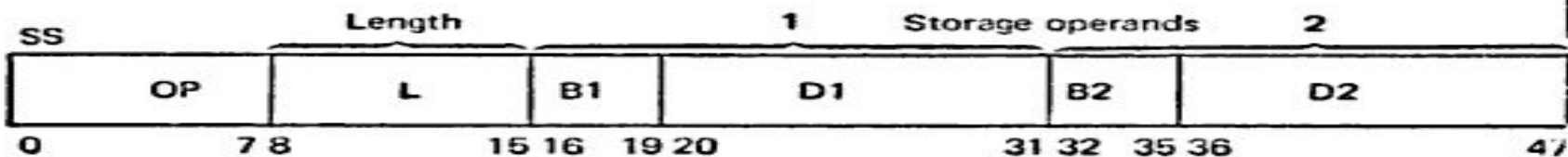
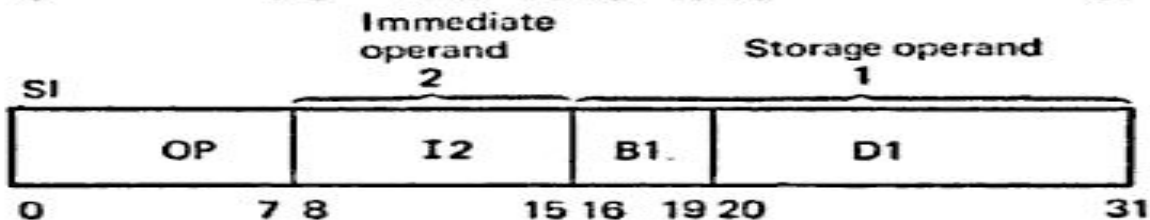
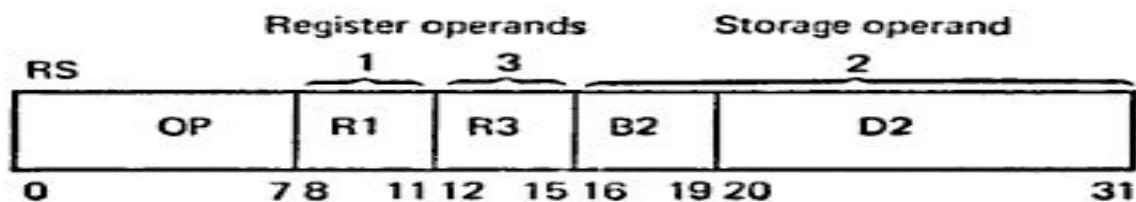
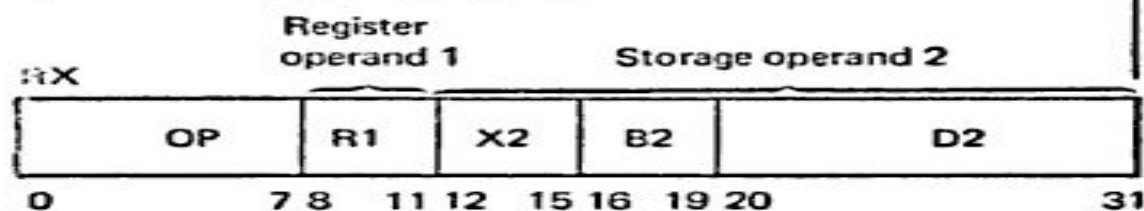
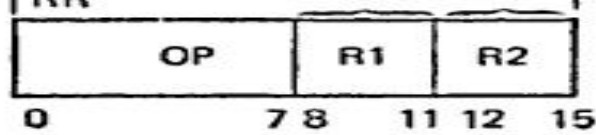
The 360 has arithmetic, logical, control or transfer, and special interrupt instructions.

The formats of the 360 instructions are depicted in Figure 2.5.

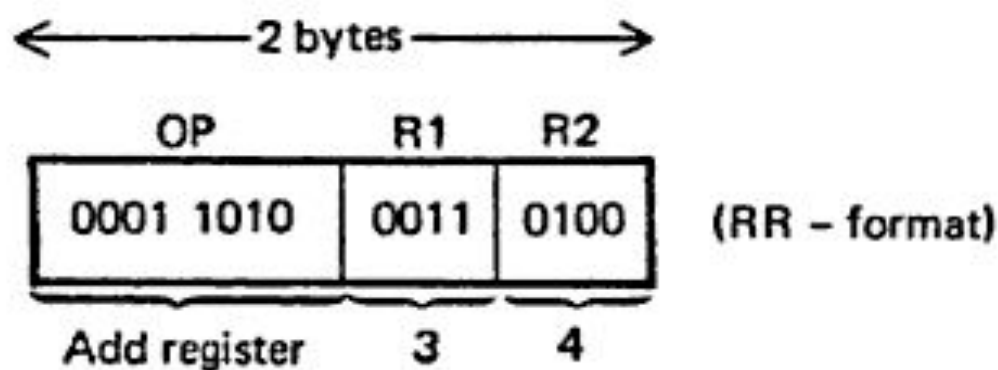
These five types of instructions differ basically in the types of operands they use.

Register operands refer to data stored in one of the 16 general registers (32 bits long), which are addressed by a four-bit field in the instruction. Since registers are usually constructed of high-speed circuitry, they provide faster access to data than does core storage.

For example, the instruction *Add register 3, 4*

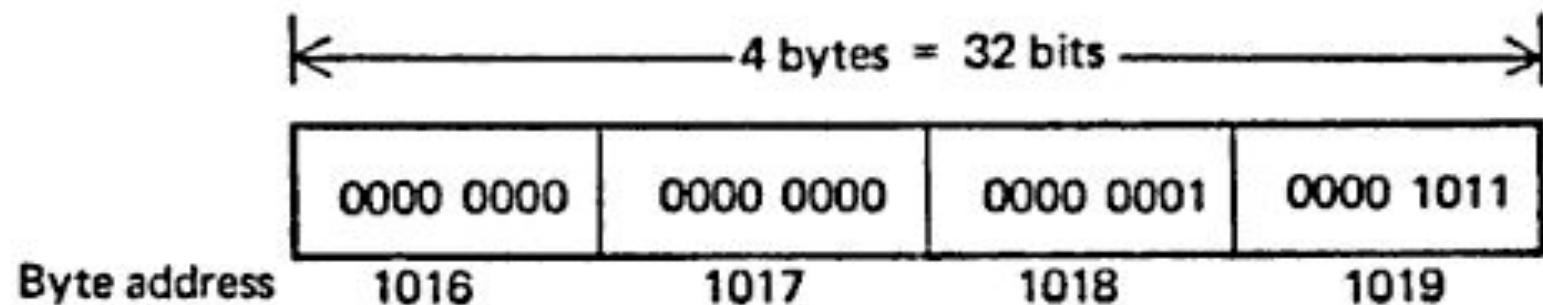


Mnemonics used:



causes the contents of general register 4 (32 bits) to be added to the contents of general register 3 (32 bits) and the resulting sum to be left in general register 3.

Storage operands refer to data stored in core memory. The length of the operand depends upon the specific data type (as illustrated in Figure 2.3). Operand data fields that are longer than one byte are specified by the address of the lowest-address byte (logically leftmost). For example, the 32-bit binary fixed-point full word with value +267 (in hexadecimal X'00 00 01 0B'), stored in locations 1016, 1017, 1018, and 1019 as depicted below is said to be "located at address 1016."



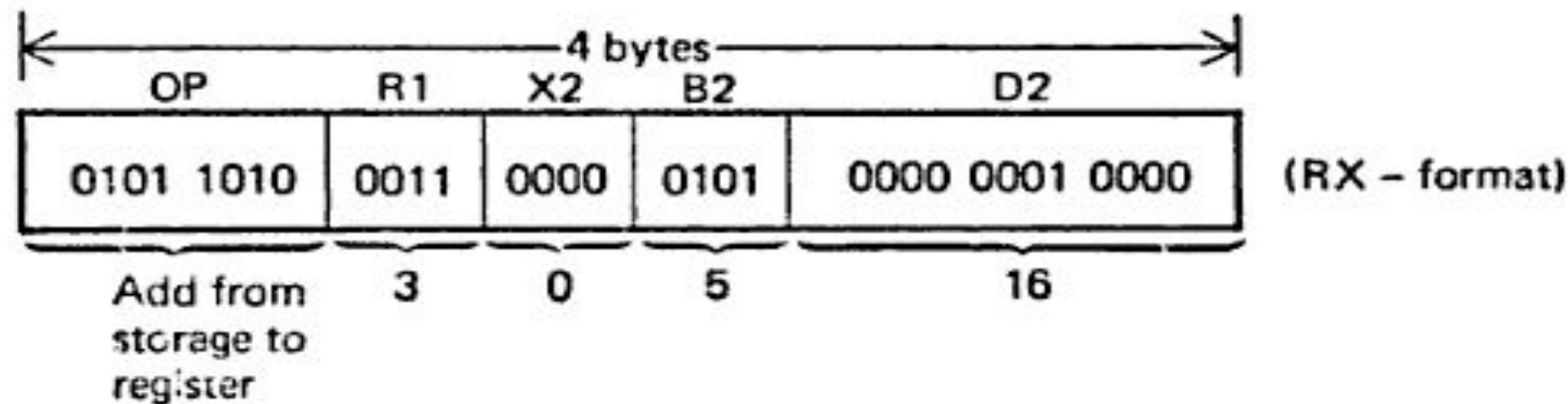
The address of the i^{th} storage operand is computed from the instruction in the following way:

$$\begin{array}{ll} \text{Address} = c(B_i) + c(X_i) + DI & \text{(RX format)} \\ \text{or} \quad c(B_i) + DI & \text{(RS, SI, SS format)} \end{array}$$

where $c(B_i)$ and $c(X_i)$ denote the contents of general registers B_i and X_i respectively. Exception: if $X_i=0$, then $c(X_i)$ are treated as 0; likewise for $B_i=0$.

For example, if we assume that general register 5 contains the number 1000, the following instruction:

For example, if we assume that general register 5 contains the number 1000, the following instruction:

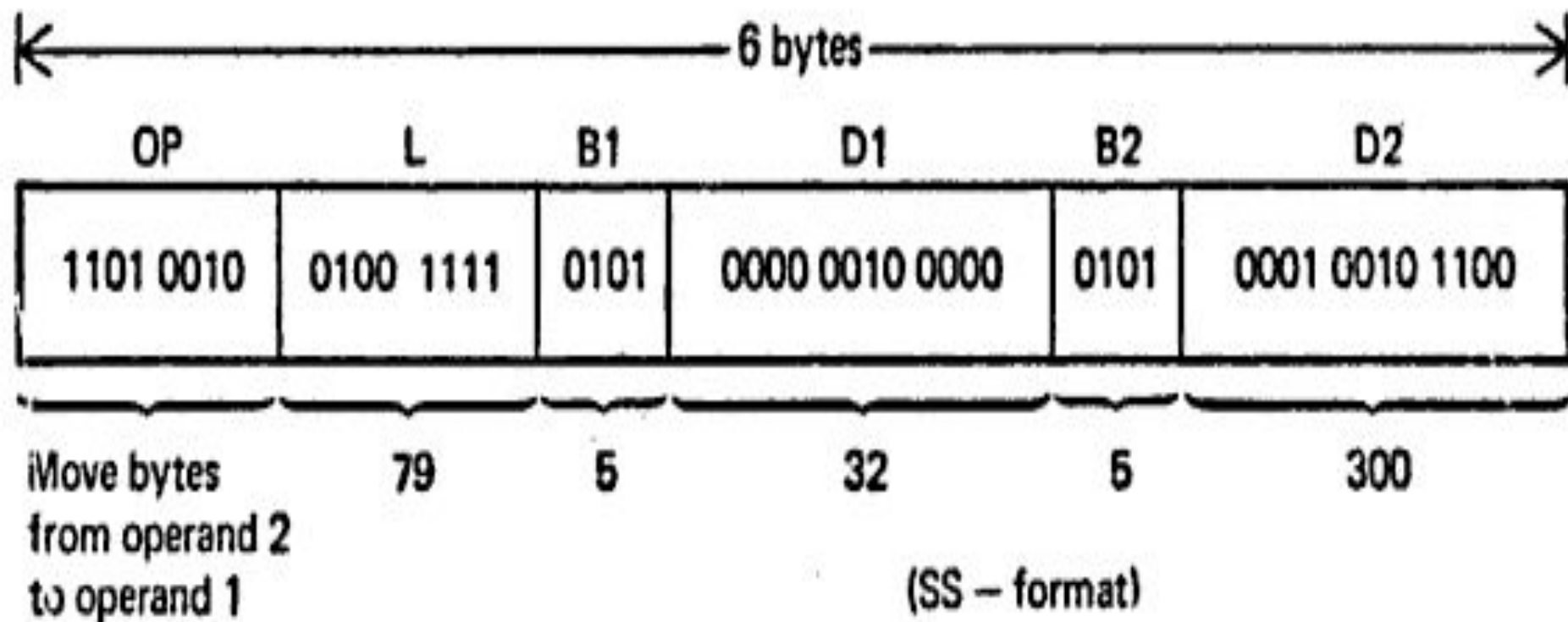


causes the contents of the word (32 bits) located at address

$$\begin{aligned}
 &= c(B2) + c(X2) + D2 \\
 &= c(5) + c(0) + 16 \\
 &= 1000 + 0 + 16 \\
 &= 1016
 \end{aligned}$$

to be added to the contents of general register 3 (32 bits) with the resulting sum

Another example, assuming again that general register 5 contains 1000, is the following instruction: (Note: in SS instructions the length is always one less than the data moved, e.g., length = 0 means move one byte.)



This instruction involves two storage operands:

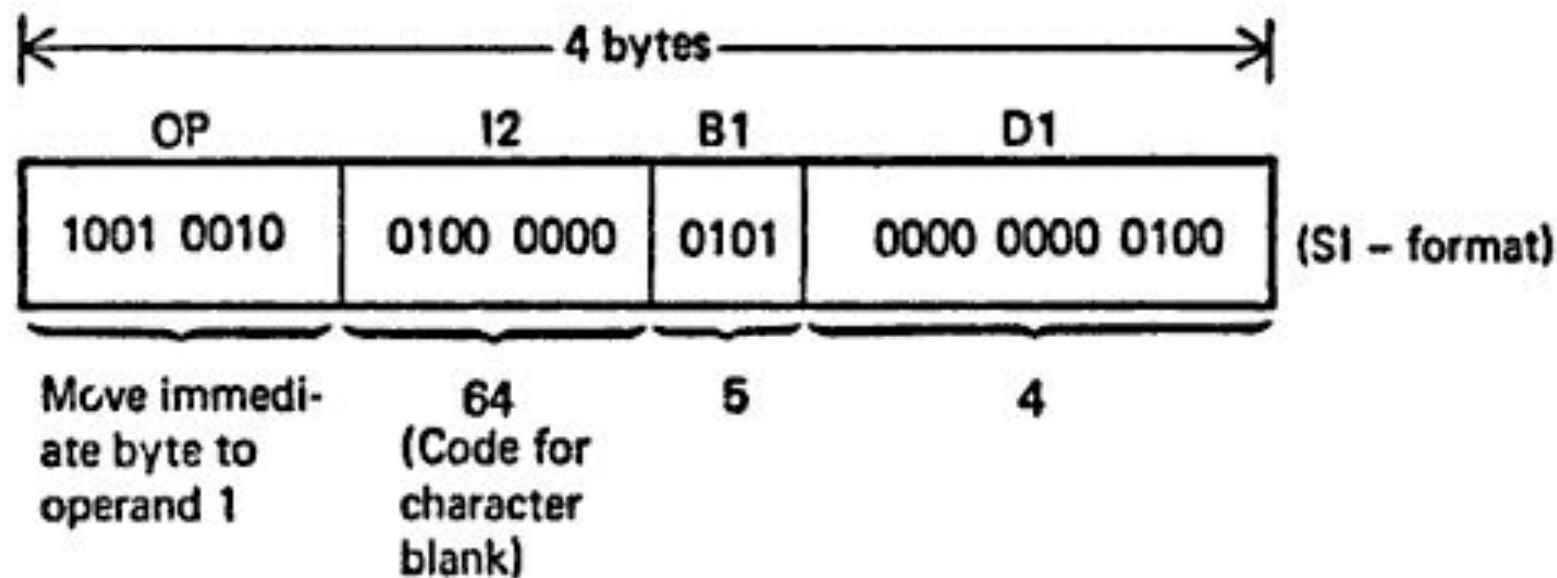
$$\begin{aligned}\text{Storage operand 1 address} &= c(B1) + D1 = c(5) + 32 \\ &= 1000 + 32 = 1032\end{aligned}$$

$$\begin{aligned}\text{Storage operand 2 address} &= c(B2) + D2 = c(5) + 300 \\ &= 1000 + 300 = 1300\end{aligned}$$

This instruction copies (moves) the 80 bytes from locations 1032 - 1111 to locations 1300 - 1379. Since a character is stored as a byte (see Fig. 2.3g), this instruction could be viewed as copying an 80-character "card image" from one area to another.

Immediate operands are a single byte of data and are stored as part of the instruction.

Again assuming register 5 to contain 1000, the following SS instruction, causes the byte 0100 0000 (bits 8 through 15 of instruction) to be stored at location 1004.



Load-store-register

*Hexadecimal
op code*

Mnemonic

Meaning (format)

Load group

58

L

Load (RX)

48

LH

Load halfword (RX)

98

LM

Load multiple (RS)

18

LR

Load (RR)

12

LTR

Load and test (RR)

Store group

50

ST

Store (RX)

40

STH

Store halfword (RX)

90

STM

Store multiple (RS)

Fixed-point arithmetic

5A

4A

1A

Add-group

A

AH

AR

Add (RX)

Add halfword (RX)

Add (RR)

Compare-group

59

49

19

C

CH

CR

Compare (RX)

Compare halfword (RX)

Compare (RR)

Divide-group

5D

1D

D

DR

Divide (RX)

Divide (RR)

Fixed-point arithmetic

Multiply-group

5C

M

Multiply (RX)

4C

MH

Multiply halfword (RX)

1C

MR

Multiply (RR)

Subtract-group

5B

S

Subtract (RX)

4B

SH

Subtract halfword (RX)

1B

SR

Subtract (RR)

Compare-group

55	CL	Compare logical (RX)
D5	CLC	Compare logical (SS)
95	CLI	Compare logical (SI)
15	CLR	Compare logical (RR)

Move-group

D2	MVC	Move (SS)
92	MVI	Move (SI)

And-group

54	N	Boolean AND (RX)
D4	NC	Boolean AND (SS)
94	NI	Boolean AND (SI)
14	NR	Boolean AND (RR)

Or-group

56	O	Boolean OR (RX)
D6	OC	Boolean OR (SS)
96	OI	Boolean OR (SI)
16	OR	Boolean OR (RR)

Exclusive-or group

57	X	Exclusive-or (RX)
D7	XC	Exclusive-or (SS)
97	XI	Exclusive-or (SI)
17	XR	Exclusive-or (RR)

Shift

8D	SLDL	Shift left (double logical) (RS)
89	SLL	Shift left (single logical) (RS)
8C	SRDL	Shift right (double logical) (RS)
88	SRL	Shift right (single logical) (RS)

*Hexadecimal
op code*

Mnemonic

Meaning (format)

Linkage group

45

BAL

Branch and link (RX)

05

BALR

Branch and link (RR)

Branch group

47

BC

Branch on condition (RX)

07

BCR

Branch on condition (RR)

46

BCT

Branch on count (RX)

06

BCTR

Branch on count (RR)

Transfer

Miscellaneous

9E
41
9C
0A
9D
43
91
42

Miscellaneous

HIO
LA
SIO
SVC
TIO
IC
TM
STC

Halt I/O (RX)
Load address (RX)
Start I/O (RX)
Supervisor call (SI)
Test I/O (RX)
Insert character (RX)
Test under mask (SI)
Store character (RX)

that transfers data between the MBR and the core memory location the address of which is in the MAR.

The *I/O channels* may be thought of as separate computers which interpret special instructions for inputting and outputting information from the memory.

To illustrate how these components of the machine structure interact, let us consider a simple computer (SC-6251). The SC-6251 has four general registers, designated 00, 01, 10, and 11 in binary rotation. The basic instruction format is as follows:

Operation code	Register number	Memory location
(op)	(reg)	(addr)